

C++ code:

```
#include<iostream>
#include<cmath>
using namespace std;

class Point {
private: double x, y;
public:
    Point(double X = 0, double Y = 0) : x(X), y(Y) { cout << "構造了一個Point類對象" << endl; }
    Point(const Point& p) : x(p.x), y(p.y) { cout << "複製構造了一個Point類對象" << endl; }
    ~Point() { cout << "析構了一個Point類對象" << endl; }
    void Set(const double X, const double Y) { x = X; y = Y; }
    void Print() { cout << "座標 " << "(" << x << "," << y << ")"; }
    friend double divX(const Point& p1, const Point& p2) { return abs(p1.x - p2.x); }
    friend double divY(const Point& p1, const Point& p2) { return abs(p1.y - p2.y); }
};

class Rectangle {
private: Point p_l, p_r;
public:
    Rectangle(Point p1 = (0,0), Point p2 = (0,0)) : p_l(p1), p_r(p2){ cout << "構造了一個Rectangle類對象" << endl; }
    Rectangle(const Rectangle& R) : p_l(R.p_l), p_r(R.p_r){ cout << "複製構造了一個Rectangle類對象" << endl; }
    ~Rectangle() { cout << "析構了一個Rectangle類對象" << endl; }
    void Set(const Point& p1, const Point& p2) { p_l = p1; p_r = p2; }
    void Print() { cout << "左上角的"; p_l.Print(); cout << " 右下角的"; p_r.Print(); }
    double Area() { return divX(p_l, p_r) * divY(p_l, p_r); }
};

int main() {
    Point P1_Left(1, 2), P1_Right(3, 4), P2_Left, P2_Right;
    Rectangle Rec1(P1_Left, P1_Right), Rec2(Rec1), Rec3;
    P2_Left.Set(9, 7);
    P2_Right.Set(15, 4);
    Rec3.Set(P2_Left, P2_Right);
    cout << "第一個矩形"; Rec1.Print(); cout << " 面積 = " << Rec1.Area() << endl;
    cout << "第二個矩形"; Rec2.Print(); cout << " 面積 = " << Rec2.Area() << endl;
    cout << "第三個矩形"; Rec3.Print(); cout << " 面積 = " << Rec3.Area() << endl;
    return 0;
}
```

運行結果:



2.

C++ code:

```
#include<iostream>
#include<cstdlib>
using namespace std;
#define pi acos(-1.0)

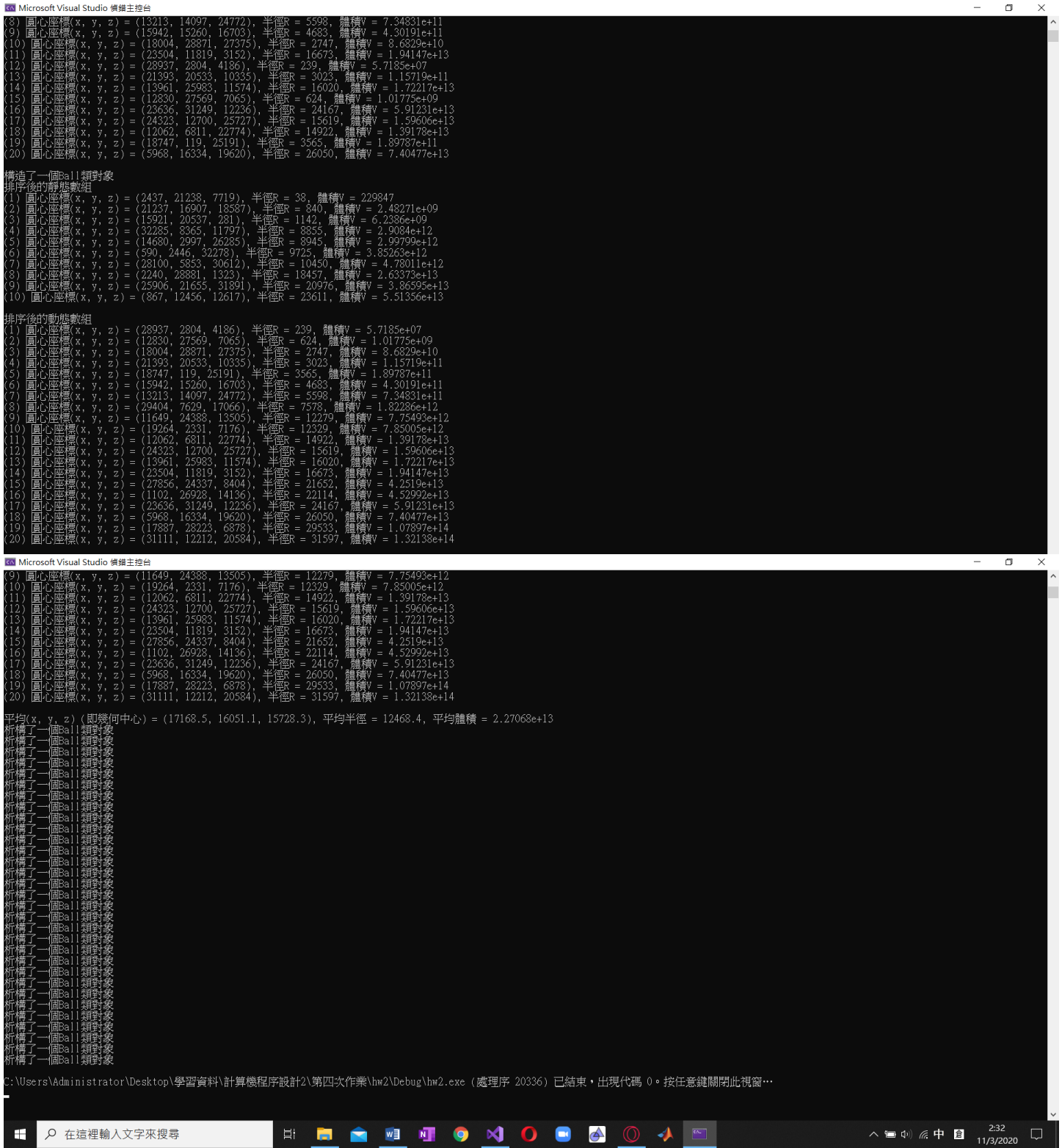
class Ball {
private:
    int x, y, z;
    int radius;
public:
    Ball(int xx = 0, int yy = 0, int zz = 0, int r = 0) :x(xx), y(yy), z(zz), radius(r) { cout << "構造了一個Ball類對象" << endl; }
    Ball(const Ball& B) :x(B.x), y(B.y), z(B.z), radius(B.radius) { cout << "複制構造了一個Ball類對象" << endl; }
    ~Ball() { cout << "析構了一個Ball類對象" << endl; }
    void Set(int xx, int yy, int zz, int r) { x = xx; y = yy; z = zz; radius = r; }
    int GetX() { return x; }
    int GetY() { return y; }
    int GetZ() { return z; }
    int GetR() { return radius; }
    double Volume() { return 4.0 / 3.0 * pi * (double)radius * (double)radius * (double)radius; }
    void Print() { cout << "圓心座標(x, y, z) = (" << x << ", " << y << ", " << z << "), " << "半徑R = " << radius; }
};

int main() {
    static Ball A[10];
    Ball* B = new Ball[20];
    srand(0);
    for (int i = 0; i < 10; i++)
        A[i].Set(rand(), rand(), rand(), abs(rand()));
    for(int i = 0; i < 20; i++)
        B[i].Set(rand(), rand(), rand(), abs(rand()));

    int sum_x = 0, sum_y = 0, sum_z = 0, sum_r = 0;
    double sum_V = 0.0;
    cout << "排序前的靜態數組" << endl;
    for (int i = 0; i < 10; i++) {
        cout << "(" << i + 1 << ") ";
        A[i].Print();
        cout << ", 體積V = " << A[i].Volume() << endl;
        sum_x += A[i].GetX();
        sum_y += A[i].GetY();
        sum_z += A[i].GetZ();
        sum_r += A[i].GetR();
        sum_V += A[i].Volume();
    }
    cout << endl;

    cout << "排序前的動態數組" << endl;
    for (int i = 0; i < 20; i++) {
        cout << "(" << i + 1 << ") ";
        B[i].Print();
        cout << ", 體積V = " << B[i].Volume() << endl;
        sum_x += B[i].GetX();
        sum_y += B[i].GetY();
        sum_z += B[i].GetZ();
        sum_r += B[i].GetR();
        sum_V += B[i].Volume();
    }
    cout << endl;

    Ball t;
```

3.

C++ code: (輸入-1 時代表輸入結束)

```
#include<iostream>
using namespace std;

class List;
class ListNode {
private:
    int data;
    ListNode* next;
public:
    ListNode(int D = 0, ListNode* N = NULL) :data(D), next(N){ cout << "構造了一個ListNode類對象" << endl; }
    ListNode(const ListNode& L) :data(L.data), next(L.next) { cout << "複制構造了一個ListNode類對象" << endl; }
    ~ListNode() { cout << "析構了一個ListNode類對象" << endl; }
    friend class List;
};

class List {
private:
    ListNode* first, * current;
public:
    List() { first = NULL; current = NULL; cout << "構造了一個List類對象" << endl; }
    ~List() { cout << "析構了一個List類對象" << endl; }
    bool Ins(int DATA, int pos) {
        ListNode* newNode = new ListNode(DATA, NULL);
        if (pos >= 1) {
            current = first;
            for (int i = 0; i < pos - 1; i++) {
                if (current == NULL) break;
                else current = current->next;
            }
            if (current == NULL) {
                cout << "Insert Error!!" << endl; return false;
            }
            else {
                newNode->next = current->next;
                current->next = newNode;
                return true;
            }
        }
        else {
            newNode->next = first;
            first = newNode;
            current = first;
            return true;
        }
    }
    bool Rem(int &del, int pos) {
        current = first;
        if (pos >= 1) {
            for (int i = 0; i < pos - 1; i++) {
                if (current == NULL) break;
                else current = current->next;
            }
            if (current == NULL || current->next == NULL) {
                cout << "Remove Error!!" << endl; return false;
            }
            else {
                ListNode* delNode = current->next;
                del = delNode->data;
                current->next = delNode->next;
                delete delNode;
                return true;
            }
        }
        else {
```

```

        if (current == NULL) {
            cout << "Remove Error!!" << endl; return false;
        }
        else {
            ListNode* delNode = first;
            del = delNode->data;
            first = delNode->next;
            delete delNode;
            return true;
        }
    }
}

};

int main() {
    int input, num1, n;
    List NUM;
    cin >> input;
    for (n = 0; input > 0; n++) {
        for (int i = 0; i <= n; i++) {
            if (i < n) {
                NUM.Rem(num1, i);
                NUM.Ins(num1, i);
                if (input <= num1) {
                    NUM.Ins(input, i); break;
                }
            }
            else {
                NUM.Ins(input, i); break;
            }
        }
        cin >> input;
    }
    for (int i = 0; i < n; i++) {
        NUM.Rem(num1, 0); cout << num1 << " ";
    }
    cout << endl;
    return 0;
}

```

運行結果:

The screenshot shows the Microsoft Visual Studio IDE with the console window open. The console displays the output of the program, which is a sequence of numbers: 56 18 320 247 12 16 13 18 -1. Below the output, there are numerous messages indicating the creation and destruction of objects, such as "構造了一個ListNode類對象" (Constructed a ListNode class object) and "析構了一個ListNode類對象" (Destructed a ListNode class object). The messages are repeated for each number in the output sequence, showing the dynamic nature of the linked list operations.